

数字抽象、CMOS 与组合逻辑基础

第三章第一讲：从 MOS 器件到逻辑门，再到组合逻辑与时间直觉

Digital Circuit Getting Started

2026 年 5 月 22 日

Learning Objectives

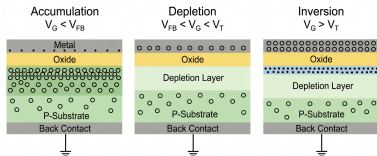
- 理解逻辑 0/1 不是天降符号，而是器件、电压阈值、噪声容限共同维持的工程抽象
- 建立 MOS、CMOS 反相器、NAND/NOR 与组合逻辑之间的递进关系
- 理解门延迟、fanout、transition 会沿路径耦合传播，时间问题从第一讲就已经出现
- 看懂组合逻辑中的竞争与冒险，并能用半加器把表达式、真值表和门级结构接起来

The Game Plan

- ① 为什么数字抽象在物理世界里成立
- ② MOS 管如何组成上拉 / 下拉网络
- ③ 反相器为什么是数字世界的基元
- ④ NAND / NOR 怎样从晶体管图走向门电路与版图对象
- ⑤ 为什么组合逻辑也会在时间上出问题

这一讲先解决“逻辑从哪里来”；下一讲再解决“系统为什么会因为时间而失败”。

Why Digital Abstraction Works

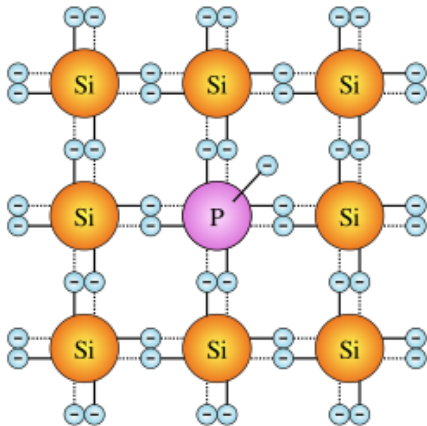


连续电压和电流最终要落到真实器件行为上

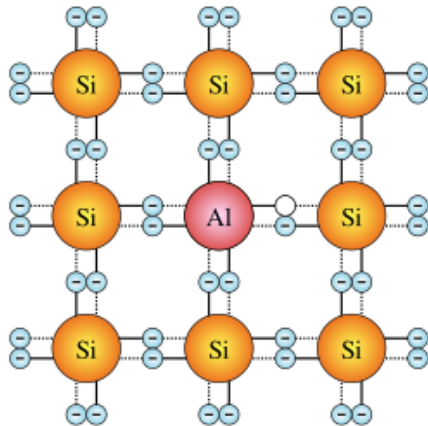
- 真实电路处理的是连续电压、电流、延迟与噪声
- 数字设计把它组织成离散的 0 / 1 与受控的状态更新
- 这层抽象成立的前提不是“世界本来就数字化”
- 而是：电平可分、噪声可容、延迟可控、边界明确

后面的 **VTC**、**setup/hold**、**metastability**，都是在保护这层抽象。

MOS Minimal Intuition: Gate-Controlled Switch



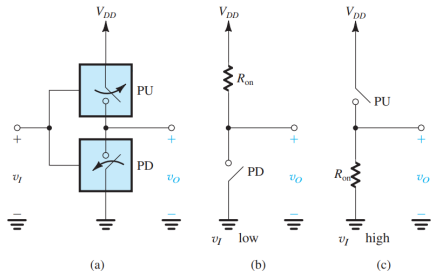
NMOS: 擅长下拉，输入高时更容易导通



PMOS: 擅长上拉，输入低时更容易导通

单个 MOS 不是理想开关，但它已经给了我们最关键的组织方式：**用栅极电压去控制电流通路是否存在。**

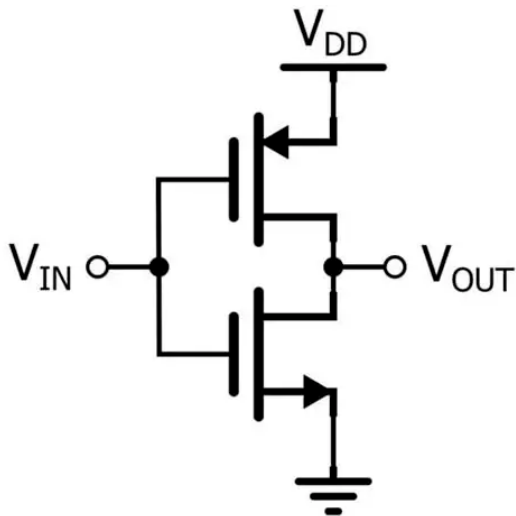
Pull-Up / Pull-Down Networks



- PMOS 组成 **pull-up network**, 把输出拉向 V_{DD}
- NMOS 组成 **pull-down network**, 把输出拉向 GND
- 静态 CMOS 的核心要求：正常情况下两张网络互补工作
- 目标不是“永远有电流流动”，而是让输出稳定停在高或低

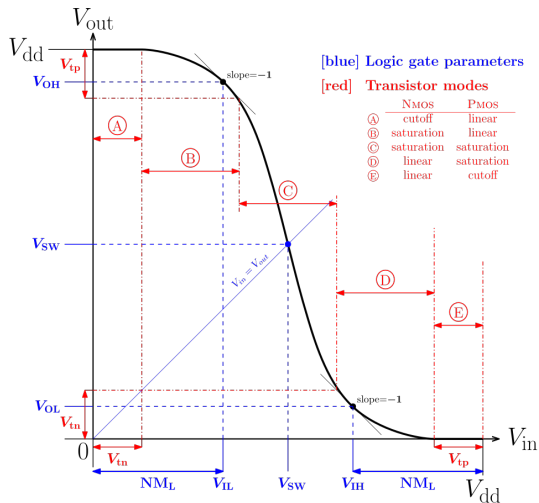
门电路不是符号游戏，而是导通路径的组织问题。

The Inverter: Smallest Complete Digital Gate



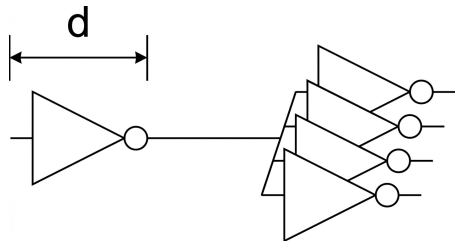
- 输入低：PMOS 导通、NMOS 关断，输出被上拉为 1
- 输入高：NMOS 导通、PMOS 关断，输出被下拉为 0
- 它把“输入控制导通路径”转换成“输出翻转”
- 反相器之所以基础，是因为它同时最简单、最规则、通常也最快

VTC, Noise Margin, and Why 0/1 Are Reliable



- 电压传输特性（VTC）展示输入变化如何映射到输出变化
- 两端平台区给出稳定逻辑区间，中间陡峭区是翻转区
- 噪声容限的作用：允许信号带着扰动仍被正确识别为 0 或 1
- 数字抽象依赖的不是完美电平，而是足够宽的可判别区间

Delay Comes From Real Devices

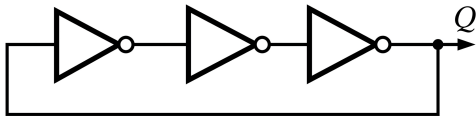


更大的负载与 fanout 会拖慢翻转

- 输出翻转要经历电容充电 / 放电，不可能瞬时完成
- 输入边沿越慢、负载越重、扇出越大，门延迟通常越大
- 上升沿和下降沿也未必对称
- 输入翻转区间内，PMOS/NMOS 可能短时间同时导通，出现短路电流

结论：门延迟不是静态常数，而是上下文相关的。

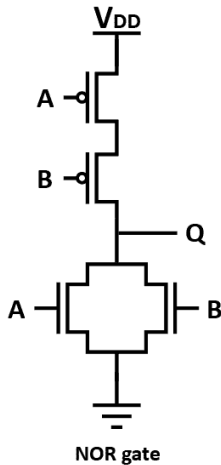
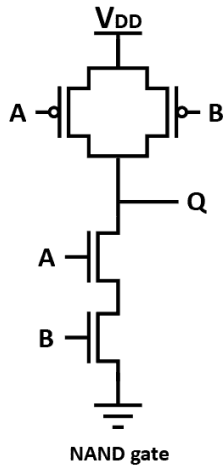
Ring Oscillator: Delay Is Not a Footnote



- 奇数个反相器首尾相接，不会收敛到静态值
- 信号会沿着反相链不断传播并翻转
- 振荡周期直接受每级延迟影响
- 这说明“延迟”不是实现细节，它会直接决定系统行为

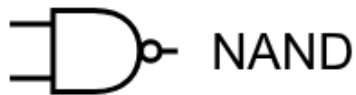
如果忽略延迟，就解释不了为什么它会振。

From Inverter to NAND / NOR



- NMOS 串联更适合表达 AND 条件下的下拉
- PMOS 并联更适合表达 OR 条件下的上拉
- 互补地组织两张网络，就得到静态 CMOS NAND / NOR
- 真值表、布尔表达式与晶体管结构是三种互相可翻译的视角

Gate Symbol, Layout View, and Standard Cell Intuition

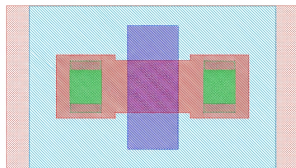


NAND

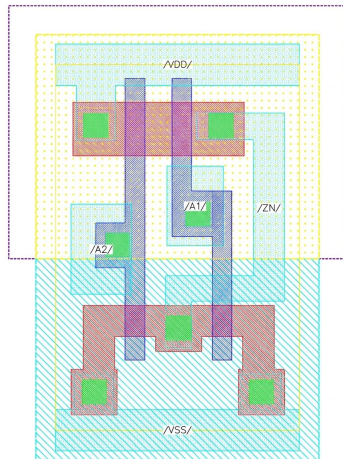


NOR

逻辑符号视角



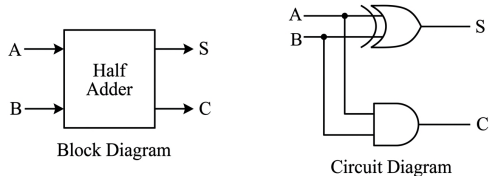
器件 / 版图视角



规则化单元视角

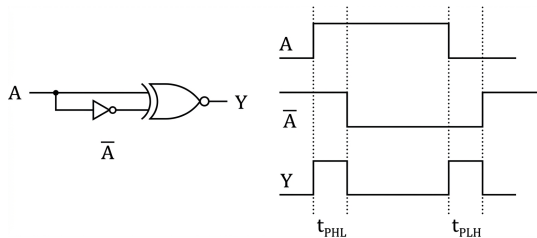
逻辑门最后会落成有边界、有引脚、有面积和速度权衡的物理对象。AOI/OAI、不同 drive strength 的单元都属于这些线的延伸

Combinational Logic Is a Current-Input / Current-Output Mapping



- 组合逻辑的定义：当前输出只依赖当前输入
- 半加器把两个输入映射成 SUM 和 CARRY
- 你可以在真值表、表达式和门级结构之间往返切换
- 全加器、译码器、比较器，都是同一组织方式的扩展

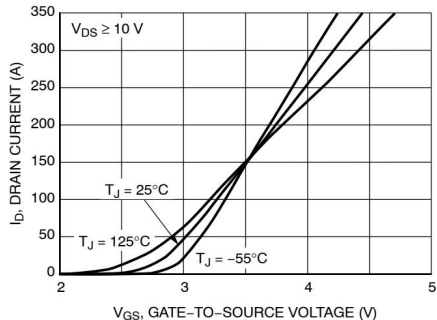
Hazard and Glitch: Why Combinational Logic Still Has Time Trouble



- 逻辑上等价，不代表时间上同时到达
- 不同路径延迟不同，就可能在输出端短暂出现毛刺
- 这类问题常被叫做竞争（race）与冒险（hazard）
- 如果毛刺被后级寄存器或异步接口看到，它就不再是“无害瞬态”

所以“功能正确”与“时间上安全”不是同一个命题。

A Small Reminder: Static CMOS Is Not the Only Way



- 传输门提醒我们：数字逻辑不只靠静态 CMOS 门组织
- 它常用于开关、复用、锁存器等结构里
- 这一讲只建立“存在其他组织方式”的视野
- 重点仍然放在最稳固的主线：器件 → 反相器 → 基础逻辑门 → 组合逻辑

Takeaways

- 数字抽象成立，是因为真实器件能把电压范围组织成可判别、可恢复的逻辑电平
- CMOS 反相器是最小完整基元，NAND / NOR 则把这种组织方式推广成可组合的逻辑门
- 门延迟、fanout、transition 与竞争/冒险说明：时间问题并不是后端才出现
- 下一讲的寄存器、setup/hold 与 metastability，本质上是在继续保护这层抽象

After-Class Prompt

请完成三件事：

- ① 画出 CMOS 反相器，分别标出输入为 0 和 1 时的导通路径
- ② 根据半加器真值表写出 SUM 与 CARRY 的表达式，并画出门级结构
- ③ 观察一个两条路径延迟不同的组合逻辑网络，解释毛刺为什么出现